Segunda entrega — Lógica secuencial

Electrónica Digital I · Sistema digital de monitoreo y control de nivel de un tanque mediante FPGA · Brahiam Campiño · Luisa Gallo · Oct 12, 2026

1. Objetivo

En esta entrega el sistema deja de ser puramente combinacional: ahora **recuerda**. El grupo debe ampliar, sobre la misma FPGA y en VHDL, el sistema de la primera entrega para que el usuario configure el llenado desde un **teclado matricial 4×4**, con **acceso por contraseña**, **modo administrador**, **llenado hasta un nivel objetivo** y una **compuerta de vaciado** automática.

Todo comportamiento con memoria (qué tecla se presionó, cuánto tiempo ha pasado, en qué modo está el sistema, si la compuerta está abierta) debe resolverse con **flip-flops, registros, contadores y máquinas de estado finito (FSM)** descritos en código.

Al terminar, el estudiante debe ser capaz de:

1. Diseñar una FSM de Moore a partir de una especificación en lenguaje natural y dibujar su diagrama de estados.
2. Sincronizar y filtrar entradas asíncronas (teclado y sensores) antes de usarlas en lógica secuencial.
3. Generar bases de tiempo a partir de un único reloj usando contadores y señales de habilitación (*clock enable*).
4. Construir un diseño jerárquico reutilizando como componentes los bloques de la primera entrega.
5. Verificar el diseño con un *testbench* autoverificable.

2. Punto de partida

La primera entrega se conserva completa y se **instancia como componente** (`port map`); no se reescribe dentro del nuevo código. Lo que cambia es quién decide encender la motobomba: ya no es una ecuación directa, sino la FSM de control.

Bloque de la entrega 1	Se reutiliza como	Cambio exigido
Etapa A (detector de nivel / error)	Componente `detector_nivel`	Su entrada ahora llega **sincronizada y filtrada** (ver sección 6)
Etapa B (selector concurrente)	Componente `selector_nivel`	Ninguno; se mantiene sin `process`
Etapa C (reparto a displays)	Componente `bin_a_display`	Debe poder mostrar también el setpoint, la cuenta regresiva del bloqueo, dígitos de contraseña (`-`) y mensajes
LED de S5	Señal directa	Ninguno
Lógica de motobomba	**Se elimina**	La reemplaza la FSM con sus interbloqueos de seguridad

Nota de consistencia: en la guía anterior el sensor del tanque principal aparece como S0 y como S5. En esta entrega se llama **S5** en todo el documento.

3. Requerimientos funcionales

El sistema arranca **bloqueado**: la planta queda en reposo seguro (bomba y compuerta apagadas) hasta que alguien inicie sesión. Cada requerimiento tiene un código (RF) que debe citarse en el informe y en los casos de prueba.

3.1 Mapa del teclado

Tecla	Función
`0` – `9`	Dígitos (contraseña, porcentaje, opciones de menú)
`A`	Modo **nivel objetivo**
`B`	**Sin uso en esta entrega**
`C`	Cancelar: detiene el proceso en curso y vuelve al menú
`D`	Solicitar **modo administrador**

Tecla	Función
`*`	Borrar el último dígito; con el campo vacío, cerrar sesión
`#`	Confirmar (Enter)

3.2 Requerimientos

Código	Requerimiento	Criterio de aceptación
RF1	**Contraseña de usuario** de 4 dígitos. Cada dígito ingresado se muestra como `-`, nunca el número.	Contraseña correcta + `#` lleva al menú. Incorrecta muestra `Err` 2 s y suma un intento.
RF2	**Bloqueo por intentos**: 3 intentos fallidos seguidos bloquean el teclado 30 s.	Durante el bloqueo se ignoran todas las teclas, los displays muestran la cuenta regresiva y el LED de alarma parpadea a 2 Hz.
RF3	**Nivel objetivo** (`A`): el usuario ingresa 25, 50, 75 o 100 y confirma con `#`.	Un valor que no sea múltiplo de 25, sea 0 o mayor que 100 se rechaza con `Err`. La bomba se apaga en el mismo ciclo en que el nivel filtrado alcanza el objetivo.
RF4	**Compuerta de vaciado**: al alcanzar 100 % (en llenado o en manual), la compuerta se abre y se mantiene abierta hasta que el nivel vuelva a 0 %.	La compuerta queda enclavada (*latch*) aunque el nivel baje de 100 %. Al cerrarse, el sistema vuelve al menú; **no** reinicia el llenado solo.
RF5	**Interbloqueo bomba–compuerta**: la bomba y la compuerta nunca pueden estar activas al mismo tiempo, en ningún modo, ni siquiera en manual.	El testbench debe comprobarlo con `assert` en todos los ciclos de reloj.
RF6	**Modo administrador** (`D` + contraseña de administrador de 4 dígitos, distinta a la de usuario). Menú: `1` cambiar contraseña de usuario · `2` bomba manual · `3` compuerta manual · `4` reconocer alarmas · `5` ver contador de vaciados.	La nueva contraseña se ingresa dos veces y solo se guarda si coinciden. En manual siguen vigentes RF5, RF8 y la regla de S5 = 0. LED de administrador encendido mientras dure la sesión.

Código	Requerimiento	Criterio de aceptación
RF7	**Cierre por inactividad**: 20 s sin presionar ninguna tecla cierran la sesión (usuario o administrador).	Un llenado automático en curso **continúa**; solo se bloquea la interfaz. Las salidas manuales sí se apagan.
RF8	**Protección de marcha en seco** (*watchdog* de bomba): si la bomba lleva 15 s encendida y el nivel no ha subido un escalón, se declara falla de bomba.	Bomba apagada, alarma enclavada, mensaje `F-b`. Solo el administrador la reconoce (opción `4`).
RF9	**Alarma de sensor**: una combinación inválida estable por más de 1 s enclava la alarma.	Bomba apagada, mensaje `F-S`. Solo el administrador la reconoce, y solo si la lectura ya es válida.
RF10	**Filtro de oleaje**: un cambio en S1–S4 solo se acepta si permanece estable 200 ms.	Pulsos más cortos que 200 ms no deben cambiar el nivel mostrado ni la bomba.

3.3 Salidas e indicadores

Salida	Significado
LED rojo S5	Hay agua en el tanque principal (igual que la entrega 1)
LED bomba / señal `bomba_en`	Orden de encendido de la motobomba (hacia la etapa de potencia)
LED compuerta / señal `compuerta_en`	Orden de apertura de la válvula de vaciado
LED administrador	Sesión de administrador activa
LED alarma	Fijo: alarma enclavada · parpadeo 2 Hz: bloqueo por intentos
4 displays de 7 segmentos	D3 = letra de modo (`L` nivel, `P pausa`, `A` admin, `b` bloqueado) · D2–D0 = valor o mensaje

3.4 Mensajes en displays

Situación	Displays (D3 D2 D1 D0)
Esperando contraseña	`b` + guiones por dígito ingresado
Entrada rechazada	`Err`
Llenando a nivel objetivo	`L` + nivel actual en %
En pausa por falta de agua (S5 = 0)	P + nivel actual en %
Vaciando	`C` + nivel actual en %
Falla de bomba / de sensor	`F-b` / `F-S`

4. Arquitectura del sistema

El diseño se organiza en tres capas: **acondicionamiento** de entradas, **decisión** (dos FSM) y **salidas**. La Figura 1 es una arquitectura de referencia; el grupo puede proponer otra, pero debe respetar R4 (dos FSM) y R13 (jerarquía).

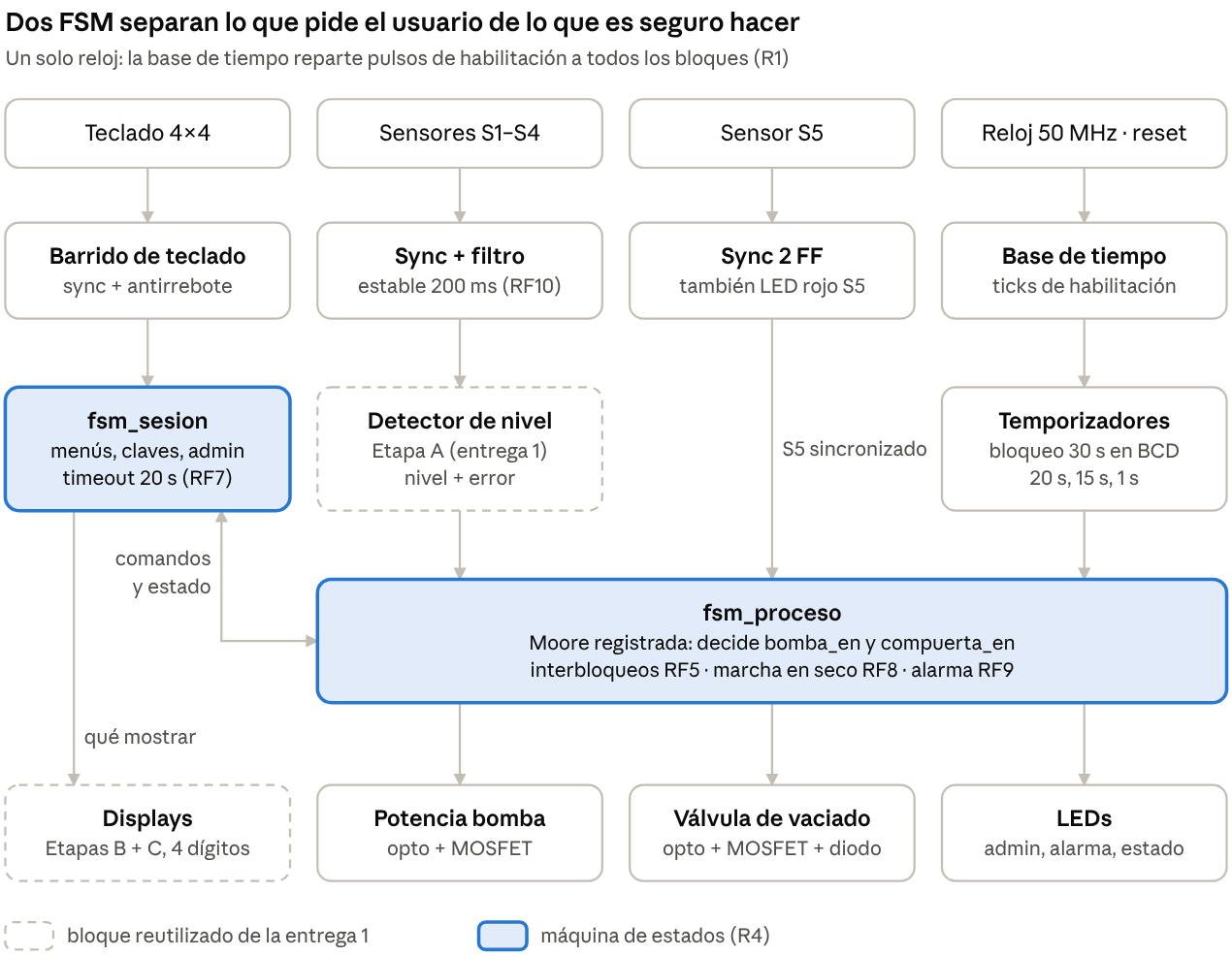


Figura 1 · Arquitectura jerárquica de referencia

`fsm_sesion` traduce teclas en **comandos** (por ejemplo, "llenar a 75 %" o "abrir compuerta en manual"). `fsm_proceso` los ejecuta solo si las condiciones físicas lo permiten. Así, ningún error en los menús puede encender la bomba en seco ni abrir la compuerta con la bomba encendida.

Entidades mínimas esperadas: `top`, `sincronizador`, `antirrebote`, `barrido_teclado`, `base_tiempo`, `temporizador_bcd`, `fsm_sesion`, `fsm_proceso`, `reg_desplazamiento`, `detector_nivel`, `selector_nivel`, `bin_a_display`.

5. Máquinas de estado

5.1 fsm_proceso (se entrega como referencia)

La Figura 2 fija el comportamiento mínimo de la planta. El grupo debe implementarla, completar su tabla de transiciones con **prioridades** y justificar cada una.

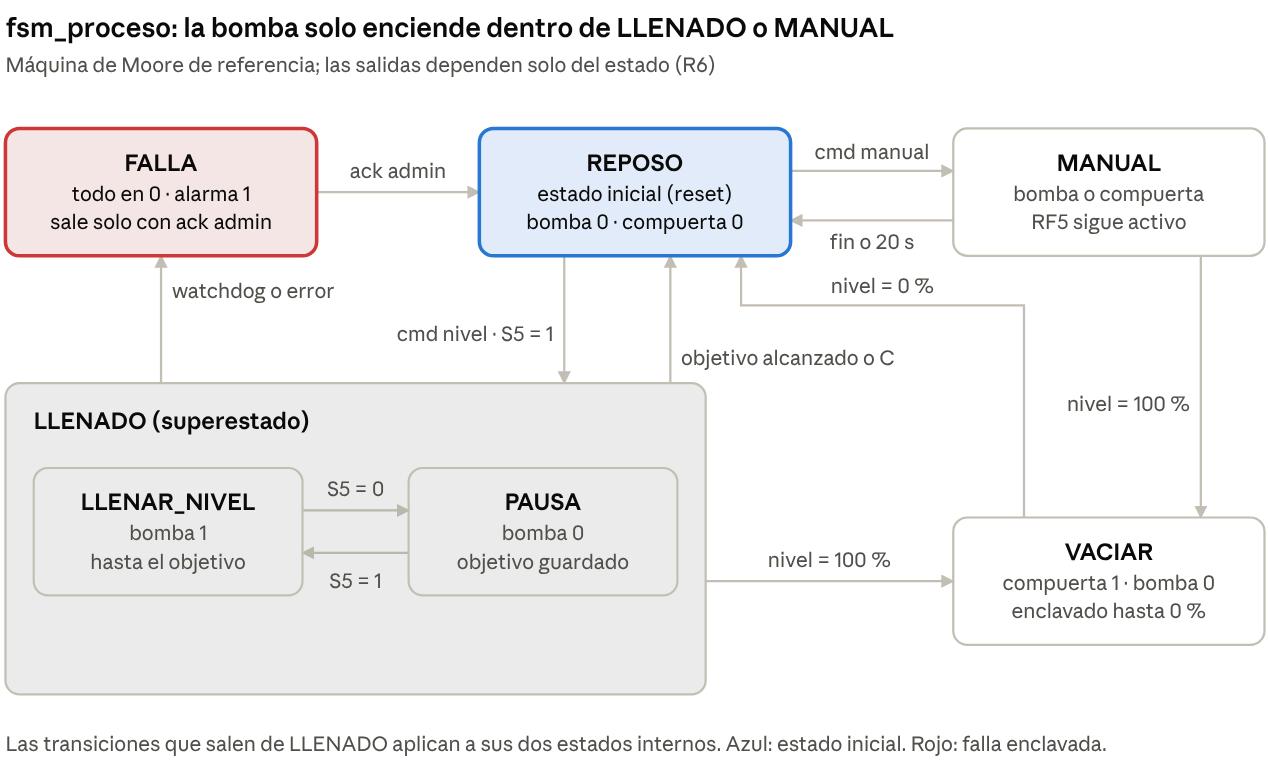


Figura 2 · Diagrama de estados de referencia de fsm_proceso

Prioridad obligatoria cuando varias condiciones se cumplen en el mismo ciclo: **1)** falla (watchdog o sensor) → **2)** nivel = 100 % → **3)** S5 = 0 → **4)** comandos del usuario. Ejemplo: si el nivel llega a 100 % justo cuando el usuario presiona `C`, gana VACIAR.

Preguntas que el informe debe responder:

1. Al salir de PAUSA el llenado continúa sin que el usuario repita el objetivo: ¿qué registro de fsm_proceso lo conserva y cuándo se carga?
2. ¿Por qué VACIAR no puede depender de la señal "nivel = 100 %" para seguir abierta?
3. ¿Qué pasaría si `bomba_en` saliera de la lógica de siguiente estado en lugar del registro de estado?

5.2 fsm_sesion (la diseña el grupo)

Esta FSM la diseñan ustedes desde cero, con diagrama y tabla de estados. Debe tener, como mínimo, estados equivalentes a:

Estado mínimo	Qué hace
BLOQUEADO	Captura la contraseña de usuario o, tras `D`, la de administrador
VERIFICAR	Compara el registro de 16 bits con la clave correspondiente
CASTIGO	Bloqueo de 30 s tras 3 intentos fallidos (RF2)
MENU_USUARIO	Espera `A`, `C` o `*`
CAPTURA_VALOR	Recibe el porcentaje objetivo (máximo 3 dígitos) y lo valida
MENU_ADMIN	Opciones 1 a 5 de RF6
CAMBIO_CLAVE	Captura la nueva clave dos veces y compara
MENSAJE	Muestra `Err` u otro aviso durante 2 s y regresa

Tabla de interfaz mínima entre las dos FSM (pueden agregar señales):

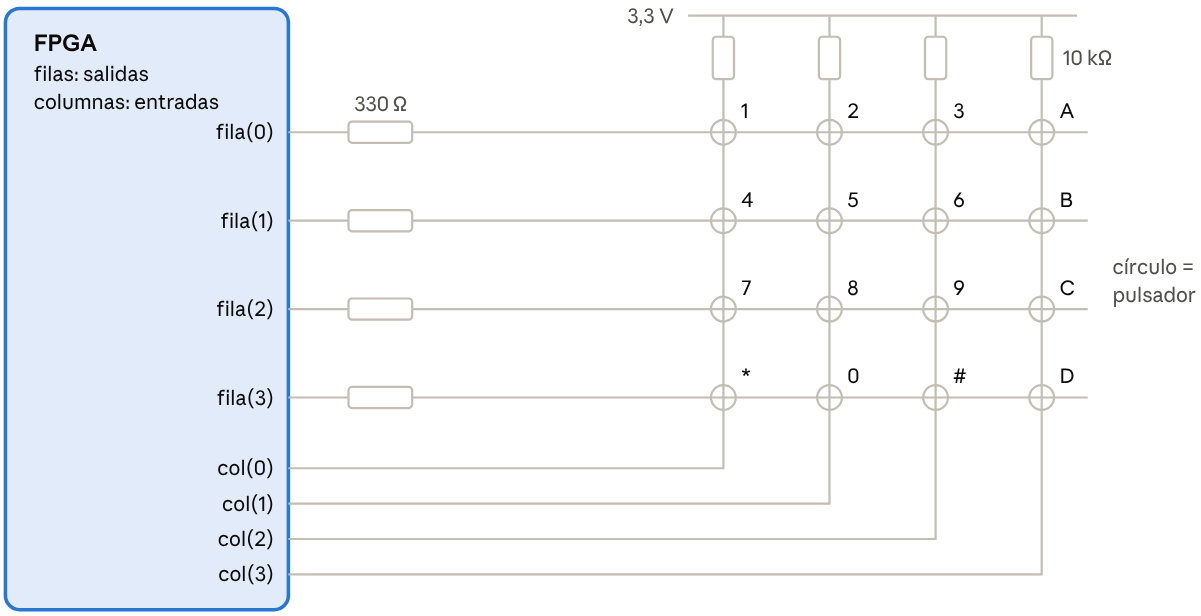
Señal	Dirección	Ancho	Significado
`cmd_nivel`	sesión → proceso	1	Pulso de un ciclo: iniciar llenado por nivel
`cmd_cancelar`	sesión → proceso	1	Pulso de un ciclo: tecla `C`
`objetivo`	sesión → proceso	3	Nivel objetivo codificado (25/50/75/100 %)
`man_bomba`, `man_comp`	sesión → proceso	1 c/u	Solicitudes manuales (nivel, no pulso)
`ack_alarma`	sesión → proceso	1	Pulso: reconocimiento de alarma por el administrador
`estado_proc`	proceso → sesión	3	Estado actual, para decidir qué mostrar

6. Interfaz del teclado matricial

El teclado no entrega un código de tecla: entrega 8 hilos. La FPGA debe **barrer** las filas, **leer** las columnas y convertir esa lectura en un pulso limpio de un ciclo con el código de la tecla.

Teclado 4×4: la FPGA baja una fila a la vez y lee qué columna cae a 0

Esquemático de referencia de protección (sección 8.1); todo a 3,3 V



Reposo: filas en 1 (o Z) y columnas en 1 por el pull-up. Barrido: una sola fila en 0; si una tecla de esa fila está presionada, su columna cae a 0. La resistencia serie limita la corriente si dos teclas unen una fila en 0 con otra en 1.

Figura 3 · Conexión y protección del teclado matricial

Cadena de procesamiento exigida, en este orden:

1. **Barrido**: un contador de 2 bits, avanzado por un *tick* de la base de tiempo (no por un reloj derivado), selecciona qué fila va a `0`. El barrido se **congela** mientras haya una tecla presionada.
2. **Sincronización**: las 4 columnas pasan por 2 flip-flops en cascada (R11).
3. **Detección**: si alguna columna está en `0`, la combinación fila + columna identifica la tecla. Si hay dos columnas en `0` a la vez, la lectura se descarta.
4. **Antirrebote**: la tecla se acepta solo si la misma lectura se mantiene estable un tiempo `T_REBOTE` (generic; valor sugerido 20 ms). Un contador se reinicia ante cualquier cambio.

5. **Pulso de tecla**: un detector de flanco genera `tecla_valida` durante **un solo ciclo** de reloj al aceptarse la tecla, junto con `tecla_cod` (4 bits). Mantener la tecla presionada no genera más pulsos.

El **filtro de oleaje** de S1–S4 (RF10) usa la misma idea del antirrebote, con 200 ms. Se espera que el grupo reutilice la misma entidad `antirrebote` con un `generic` distinto, no que la escriba dos veces.

7. Base de tiempo y temporizadores

Todo el diseño usa **un solo reloj** (R1). Los tiempos se obtienen con contadores que, al llegar a su límite, generan un *tick*: una señal en `1` durante exactamente un ciclo de reloj. Los demás bloques la usan como habilitación:

```
if rising_edge(clk) then
  if tick_1hz = '1' then
    -- aquí avanza el contador de segundos
  end if;
end if;
```

El número de cuentas de cada divisor se calcula a partir de los `generic`. Por ejemplo, para un *tick* de 1 Hz:

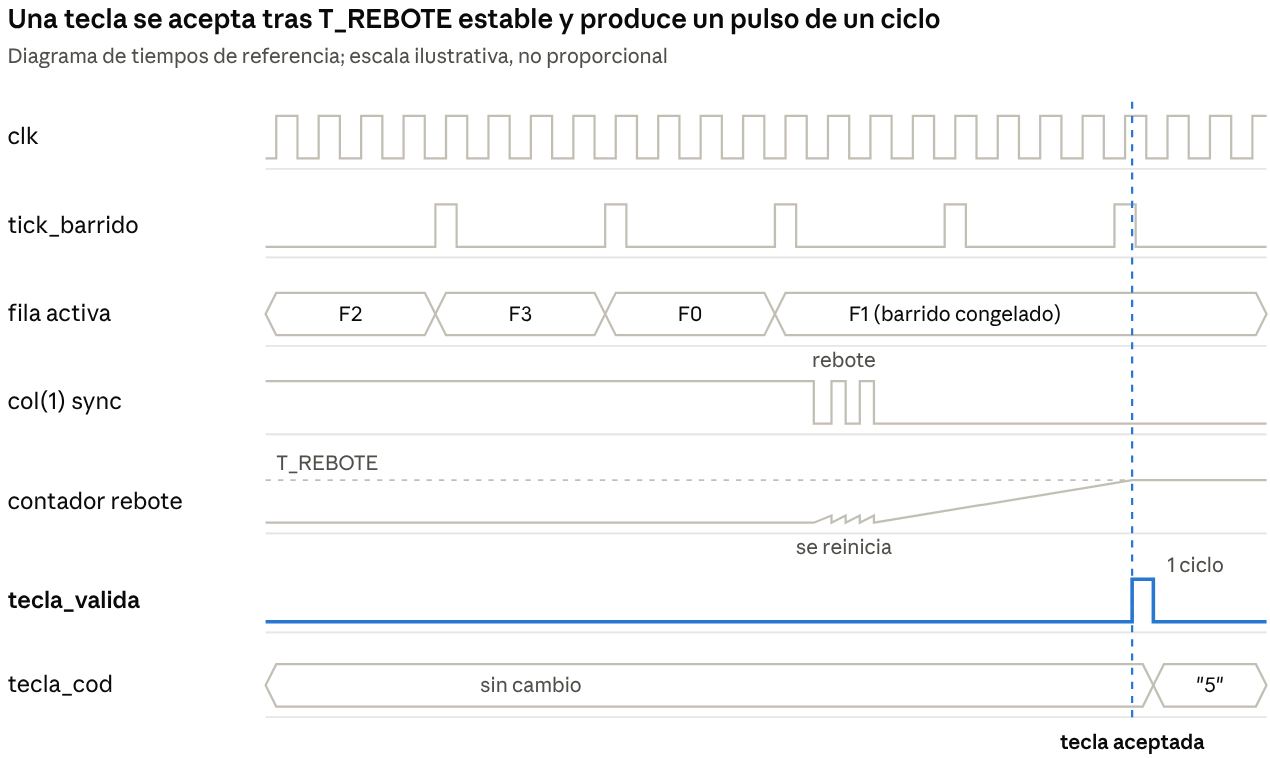
```
N_{1\,Hz} = \frac{CLK\_HZ}{1\,\mathrm{Hz}} - 1 \qquad (CLK\_HZ =
50\,\mathrm{MHz} \Rightarrow N = 49\,999\,999)
```

Temporizadores que debe tener el sistema:

Temporizador	Duración	Uso	Tipo sugerido
Antirrebote del teclado	20 ms	Paso 4 de la sección 6	Contador binario que se reinicia ante cambios
Filtro de oleaje	200 ms	RF10	Misma entidad, otro `generic`
Bloqueo por intentos	30 s	RF2	Descendente, mostrado en displays
Inactividad de sesión	20 s	RF7	Se reinicia con cada `tecla_valida`

Temporizador	Duración	Uso	Tipo sugerido
Watchdog de bomba	15 s	RF8	Se reinicia cuando el nivel sube un escalón
Confirmación de error de sensor	1 s	RF9	Se reinicia si la lectura vuelve a ser válida
Mensajes en display	2 s	`Err`, avisos	Ascendente
Parpadeo de alarma	2 Hz	RF2	Flip-flop T habilitado por un *tick* de 4 Hz

La Figura 4 muestra la relación entre el barrido, el rebote y el pulso de tecla. Su simulación debe reproducir este comportamiento.



Ejemplo: se presiona la tecla 5 (fila 1, columna 1). Los ticks siguen, pero la fila activa no cambia mientras haya una tecla presionada.

Figura 4 · Barrido, antirrebote y pulso de tecla

8. Protección de las nuevas entradas y salidas

La regla de la primera entrega sigue vigente: **ningún elemento externo se conecta directo a un pin de la FPGA**. Las nuevas conexiones son el teclado (8 pines) y la compuerta de vaciado.

8.1 Teclado matricial

- Teclado alimentado **solo a 3,3 V**; nunca a 5 V.
- **Columnas** (entradas a la FPGA) con resistencia de *pull-up* de 10 kΩ a 3,3 V. Pueden usar el *pull-up* interno de la FPGA, pero deben justificar su valor con la hoja de datos.
- **Filas** (salidas de la FPGA) con resistencia en serie de 330 Ω. Si se presionan dos teclas de la misma columna en filas distintas, una fila en `0` queda unida a otra en `1`; la resistencia limita esa corriente.
- Alternativa válida: manejar las filas en modo *drenaje abierto* (`'0'` o `'Z'`, nunca `'1'`). Si la usan, expliquen por qué elimina el cortocircuito.
- Calculen la corriente en el peor caso y compárenla con la máxima por pin de su FPGA.

8.2 Compuerta de vaciado

Se modela con una **electroválvula solenoide** (o un servo, si el grupo lo justifica). La FPGA solo entrega la señal `compuerta_en`.

- Aislamiento: optoacoplador (por ejemplo PC817) entre la FPGA y la etapa de potencia.
- Conmutación: MOSFET de nivel lógico o relé, con su resistencia de compuerta y su resistencia de *pull-down* para que la válvula quede cerrada mientras la FPGA se configura.
- **Diodo de rueda libre** en antiparalelo con la bobina de la válvula (y del relé, si se usa).
- Indiquen si la válvula es normalmente cerrada o normalmente abierta y qué pasa con el agua si se va la energía.

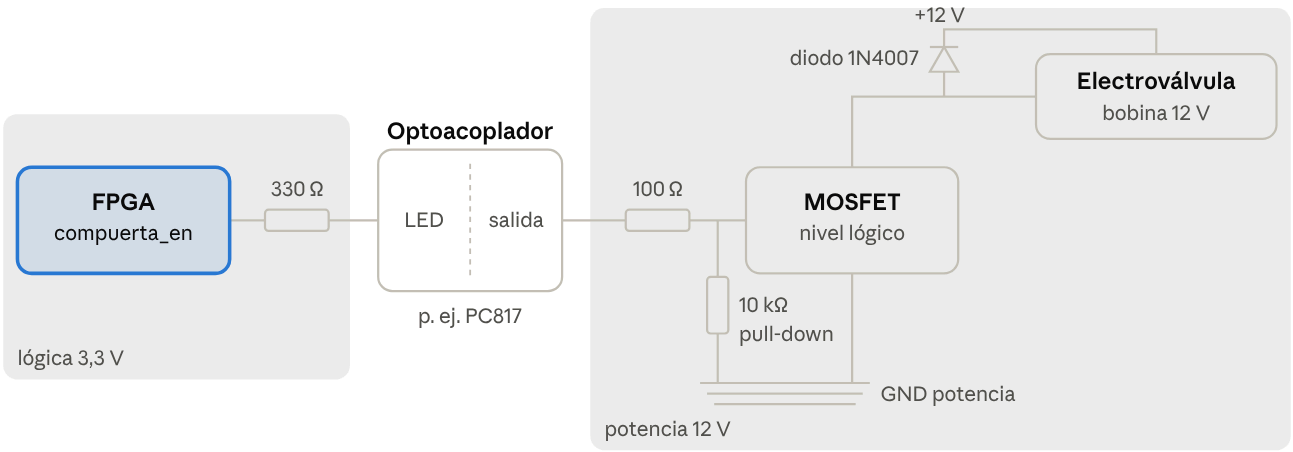


Figura 5 · Etapa de potencia de referencia para la compuerta de vaciado

8.3 LEDs nuevos

Cada LED con su resistencia limitadora calculada. Ejemplo para un LED rojo a 5 mA desde 3,3 V:

$$R = \frac{V_{CC} - V_{F}}{I_{F}} = \frac{3.3\,\mathrm{V} - 2.0\,\mathrm{V}}{5\,\mathrm{mA}} = 260\,\Omega \;\rightarrow\; 270\,\Omega$$

El esquemático del teclado está en la Figura 3 (sección 6) y el de la compuerta en la Figura 5. Ambos deben aparecer en el informe, adaptados a sus componentes reales.

9. Restricciones obligatorias de diseño en VHDL

Estas reglas no son sugerencias: un diseño que funcione pero incumpla alguna pierde los puntos de ese criterio en la rúbrica. Su propósito es que el código refleje hardware real y no un programa secuencial.

Código	Restricción	Por qué
R1	**Un solo reloj** en todo el diseño (el oscilador de la tarjeta). Ningún `rising_edge` puede aplicarse a una señal distinta de `clk`. Los divisores no generan relojes: generan pulsos *tick* de un ciclo que se usan como habilitación.	Relojes derivados de lógica producen *skew* y fallas de temporización.
R2	**Un solo reset**, síncrono y activo en bajo, tomado de un pulsador y pasado por un sincronizador de 2 flip-flops.	Evita estados indefinidos al soltar el reset.
R3	Solo las librerías `ieee.std_logic_1164` e `ieee.numeric_std`. Prohibidas `std_logic_arith` y `std_logic_unsigned`.	Son librerías no estándar con aritmética ambigua.
R4	**Dos FSM independientes**: `fsm_sesion` (interfaz y teclado) y `fsm_proceso` (planta). Solo se comunican por señales listadas en una tabla de interfaz del informe.	Separa "quién pide" de "qué es seguro hacer".
R5	Cada FSM con tipo enumerado y estilo de **dos o tres procesos** (registro de estado separado de la lógica de siguiente estado). Todo `case` termina en `when others`.	Facilita leer el diagrama desde el código y evita estados ilegales.
R6	Las salidas hacia actuadores (`bomba_en`, `compuerta_en`) son **Moore y registradas**: salen directo de un flip-flop.	Una salida combinacional puede tener *glitches* que encienden la bomba por nanosegundos.
R7	Prohibidos `/`, `mod`, `rem` y `*` **sobre señales** en código sintetizable (entre constantes y generics sí, p. ej. `CLK_HZ / 1000`: se calcula al compilar). Los segundos se cuentan con **contadores BCD en cascada**; si necesitan convertir binario a BCD, usen *double dabble* descrito por ustedes.	Obliga a construir la aritmética con registros y contadores.

Código	Restricción	Por qué
R8	Prohibidos `wait`, `after` y `variable` en código sintetizable (sí se permiten en el testbench).	Todo valor que deba recordarse es una `signal` y, por tanto, un registro visible.
R9	Las contraseñas se capturan en un **registro de desplazamiento de 16 bits** (4 dígitos × 4 bits) y se comparan completas al presionar `#`. Prohibido comparar dígito por dígito al teclear. La de usuario vive en un registro modificable; la de administrador es un `generic`.	Ejercita registros de desplazamiento y comparadores de ancho completo.
R10	Toda frecuencia y todo tiempo (`CLK_HZ`, 200 ms, 1 s, 15 s, 20 s, 30 s, 2 Hz) es un **`generic`** de la entidad *top*. Ningún número mágico dentro de las arquitecturas.	Permite simular con `CLK_HZ` pequeño sin cambiar el código.
R11	Toda entrada asíncrona (teclado y S1–S5) pasa por un **sincronizador de 2 flip-flops** antes de cualquier otra lógica.	Evita metaestabilidad.
R12	El reporte de compilación de Quartus **no puede tener latches inferidos** ni violaciones de temporización. Reporten Fmax (Timing Analyzer), número de elementos lógicos y de registros.	Un latch inferido delata un `if` o `case` incompleto.
R13	Diseño **jerárquico**: una entidad por archivo, ninguna arquitectura de más de 200 líneas, y la entidad *top* solo contiene instancias, señales de conexión y asignaciones directas de señal a puerto (sin `process` ni lógica).	Obliga a pensar en bloques, como en el diagrama de la sección 4.

Los bloques de la entrega 1 conservan sus propias restricciones (la Etapa B sigue sin `process`).

10. Plan de verificación

La simulación se hace con un **testbench escrito en VHDL** (ModelSim/Questa desde Quartus). El editor de formas de onda no es suficiente para esta entrega.

Reglas del testbench:

- **Autoverificable**: cada caso termina con `assert ... report ... severity error`. Al final se imprime cuántos casos pasaron.
- **Modelo de teclado realista**: el testbench no fuerza el código de tecla. Modela la matriz: cuando la FPGA pone en `0` la fila de la tecla "presionada", el modelo pone en `0` la columna correspondiente.
- **Rebote simulado**: cada pulsación incluye al menos 3 cambios rápidos antes de estabilizarse, y lo mismo al soltar.
- **Tiempos reducidos**: se instancia el *top* con `CLK_HZ` pequeño (por ejemplo 1 kHz) gracias a R10.
- Un `assert` concurrente vigila RF5 durante toda la simulación: `not (bomba_en = '1' and compuerta_en = '1')`.
- Se recomienda escribir un `procedure presionar_tecla(...)` dentro del testbench para no repetir código.

10.1 Casos de prueba mínimos

Caso	Verifica	Estímulo	Resultado esperado
TC1	Reset	Reset activo	FSM en estado inicial, bomba y compuerta en `0`, displays en `b`
TC2	RF1	Contraseña correcta + `#`	Paso al menú
TC3	RF1, RF2	3 contraseñas incorrectas	Bloqueo de 30 s; teclas ignoradas; desbloqueo al terminar
TC4	RF3	`A` 75 `#` con nivel 25 % y S5 = 1	Bomba encendida hasta que el nivel filtrado sea 75 %
TC5	RF3	`A` 60 `#`	`Err`; la bomba no se enciende

Caso	Verifica	Estímulo	Resultado esperado
TC6	RF3	`A 75 # con nivel 25 %; S5 cae a 0 durante el llenado y luego vuelve a 1`	Bomba apagada y D3 = P mientras S5 = 0; al volver, el llenado continúa sin reingresar el objetivo y se detiene en 75 %
TC7	RF4, RF5	Nivel llega a 100 % durante un llenado	Bomba en `0` y compuerta en `1` en el mismo flanco; compuerta abierta hasta 0 %
TC8	RF6	Cambio de contraseña con confirmación distinta y luego igual	Solo la segunda vez se guarda; la nueva contraseña funciona
TC9	RF6, RF5	En manual: bomba encendida y se pide abrir la compuerta	La bomba se apaga primero; nunca ambas en `1`
TC10	RF7	Sesión abierta sin teclas por 20 s con llenado en curso	Interfaz bloqueada; el llenado continúa
TC11	RF8	Bomba encendida con el nivel congelado 15 s	Falla `F-b`, bomba apagada, alarma enclavada
TC12	RF9, RF10	Pulso de 50 ms en S3; luego combinación inválida estable 2 s	El pulso se ignora; la combinación inválida produce `F-S`

En el informe, cada caso se presenta con su captura de formas de onda **anotada** (marcadores en los instantes clave) y una frase que diga qué demuestra.

11. Entregables y evaluación

Fecha máxima de entrega: [por definir], durante el TA. Grupos de máximo 3 personas.

Entregables:

- [ ] Código fuente VHDL completo (una entidad por archivo) y el proyecto de Quartus.
- [ ] Testbench autoverificable con los 12 casos de la sección 10.1.
- [ ] Diagrama de bloques jerárquico propio (con los nombres reales de sus entidades y señales).
- [ ] Diagrama de estados de **cada** FSM, con condiciones de transición y salidas, y su tabla de estados.
- [ ] Tabla de interfaz entre `fsm_sesion` y `fsm_proceso`.

- [ ] Esquemático de protección de teclado, compuerta y LEDs, con cálculos.
- [ ] Reporte de Quartus: elementos lógicos, registros, Fmax y evidencia de cero latches inferidos.
- [ ] Demostración en la tarjeta (sensores simulados con interruptores está permitido).
- [ ] Informe breve que justifique las decisiones de diseño y analice la simulación.

11.1 Rúbrica

Criterio	Peso
Funcionamiento en la tarjeta (RF1–RF10)	30 %
Diseño de las FSM (diagramas, tablas, correspondencia con el código)	20 %
Cumplimiento de restricciones R1–R13	15 %
Testbench y análisis de simulación	20 %
Protección de hardware y cálculos	10 %
Calidad del informe	5 %

En la sustentación, cualquier integrante debe poder explicar cualquier bloque del código.

12. Retos opcionales (bonificación)

Hasta 0,5 adicional sobre la nota de la entrega. Cada reto debe tener su propio caso de prueba.

Reto	Descripción	Bono
Zumbador con PWM	Tonos distintos para tecla aceptada, error y alarma, generados con un contador PWM.	0,1
Histéresis de nivel	En modo nivel, mantener el tanque entre el objetivo y un escalón abajo, reactivando la bomba sin intervención.	0,1
Registro por UART	Enviar a un PC cada evento (inicio de sesión, falla, apertura de compuerta) con un transmisor UART propio.	0,2

Reto	Descripción	Bono
Displays multiplexados	Manejar los 4 displays con un solo bus de segmentos y barrido, como en una tarjeta de bajo costo.	0,1
Horómetro de bomba	Contador BCD de minutos acumulados de funcionamiento, visible en el menú de administrador.	0,1